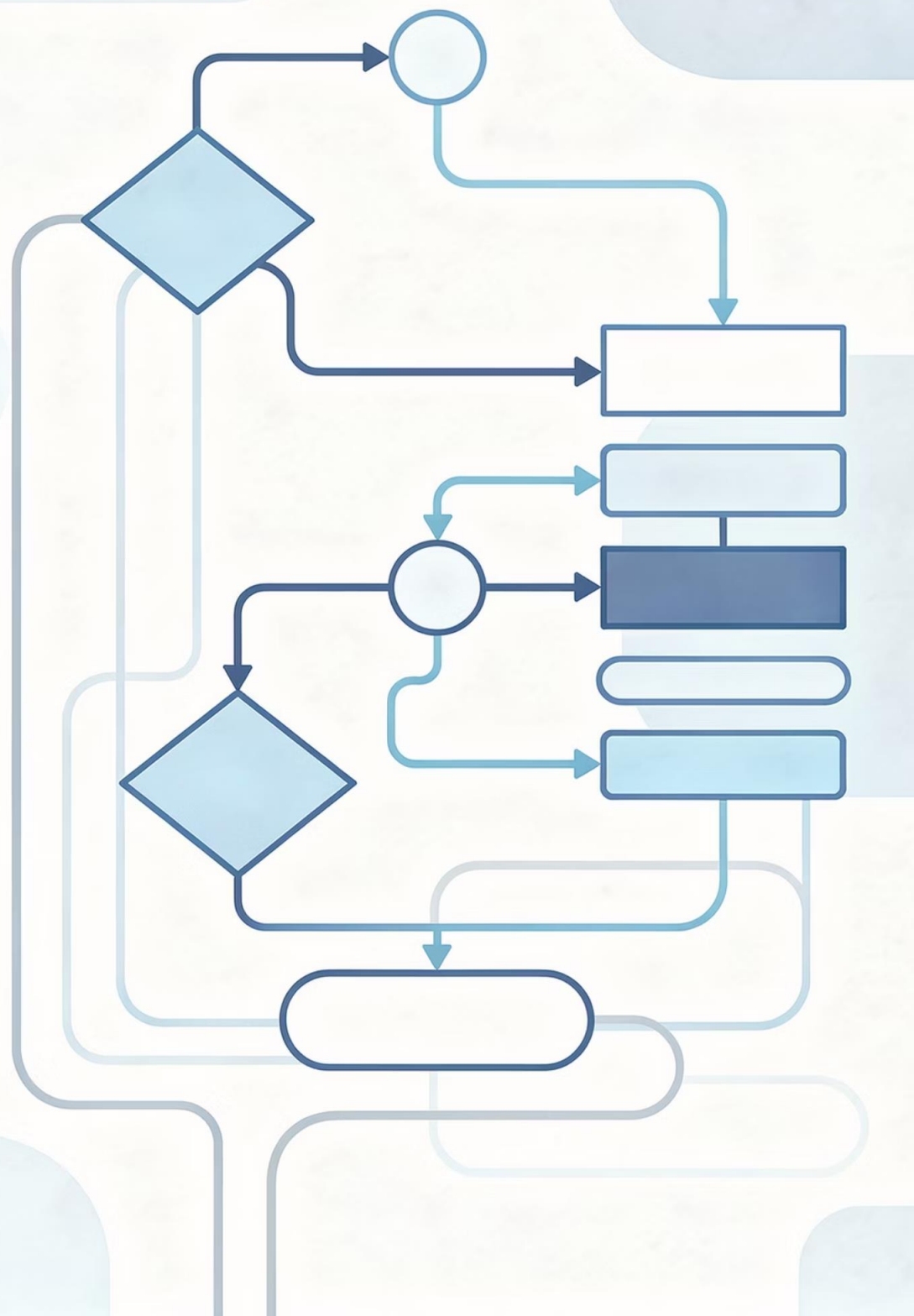




Módulo I

Lógica de Programação

Representação de Algoritmos

O Que Vamos Aprender

Entender como representar algoritmos é fundamental para o desenvolvimento de soluções computacionais eficientes. Nesta apresentação, exploraremos as principais formas de representar algoritmos, desde descrições em linguagem natural até notações mais estruturadas como o pseudocódigo e a linguagem de programação.



Compreender Representações

Conhecer as diferentes formas de representar algoritmos e suas características específicas



Aplicar Técnicas

Aprender a escolher a melhor forma de representação para cada situação de desenvolvimento



Desenvolver Lógica

Fortalecer o raciocínio lógico através da prática de expressão algorítmica

Por Que Representar Algoritmos?

Importância Fundamental

A representação de algoritmos é a ponte entre o pensamento lógico e a implementação computacional. Ela permite comunicar ideias, documentar soluções e validar raciocínios antes da codificação.

- Facilita a comunicação entre desenvolvedores e equipes técnicas
- Permite identificar erros lógicos antes da programação
- Documenta o processo de solução de problemas
- Independe de linguagens de programação específicas
- Ajuda no planejamento e organização do código

Formas de Representação

Existem diversas maneiras de representar algoritmos, cada uma com suas vantagens e aplicações específicas. As quatro formas principais são a descrição narrativa, o fluxograma, o pseudocódigo e a linguagem de programação, que complementam o processo de desenvolvimento algorítmico.

1 Descrição Narrativa (Linguagem Natural)

Uso de linguagem natural para descrever os passos

2 Fluxograma

Representação gráfica usando símbolos padronizados

3 Pseudocódigo

Notação estruturada que combina elementos de linguagens de programação

4 Linguagem de Programação

Implementação em uma linguagem específica (Python, Java, PHP, etc.)



Descrição Narrativa

O Que É?

A descrição narrativa é a forma mais simples e intuitiva de representar um algoritmo. Utiliza linguagem natural (português, no nosso caso) para descrever, passo a passo, as ações necessárias para resolver um problema.

É especialmente útil para iniciantes e para comunicar ideias de forma clara para pessoas sem conhecimento técnico aprofundado.

Características Principais

- Linguagem coloquial e acessível
- Sequência lógica e ordenada de passos
- Não requer conhecimento de programação
- Fácil compreensão por não-programadores
- Pode ser ambígua em alguns casos

Vantagens e Limitações

Descrição Narrativa

Vantagens ✓

- Fácil de entender para qualquer pessoa
- Não requer conhecimento técnico
- Ideal para comunicação inicial
- Linguagem acessível e natural

Limitações ✗

- Pode gerar ambiguidades
- Falta de precisão técnica
- Dificulta detalhamento de lógica complexa
- Não é diretamente executável

Exemplo de Descrição Narrativa

Algoritmo: Fazer um café

1

Verificar se há água no reservatório da cafeteira

2

Se não houver água suficiente, adicionar água até o nível desejado

3

Colocar o filtro de papel no suporte da cafeteira

4

Adicionar o pó de café no filtro (2 colheres por xícara)

5

Ligar a cafeteira e aguardar o café passar completamente

6

Desligar a cafeteira e servir o café

Note como a descrição narrativa usa linguagem natural e clara, tornando o processo compreensível para qualquer pessoa, mesmo sem conhecimentos de programação.

Fluxograma

O fluxograma é uma representação gráfica do algoritmo que utiliza símbolos padronizados para ilustrar o fluxo de execução. É uma ferramenta visual que facilita a compreensão da lógica e do sequenciamento das operações.

Oval

Início/Fim (Terminal)

Retângulo

Processamento (operações e cálculos)

Paralelogramo

Entrada/Saída (leitura e escrita de dados)

Losango

Decisão (estruturas condicionais)

Setas

Fluxo de execução

Vantagens

- Visual e intuitivo
- Facilita identificação de erros lógicos
- Excelente para documentação
- Útil para comunicação em equipe

Limitações

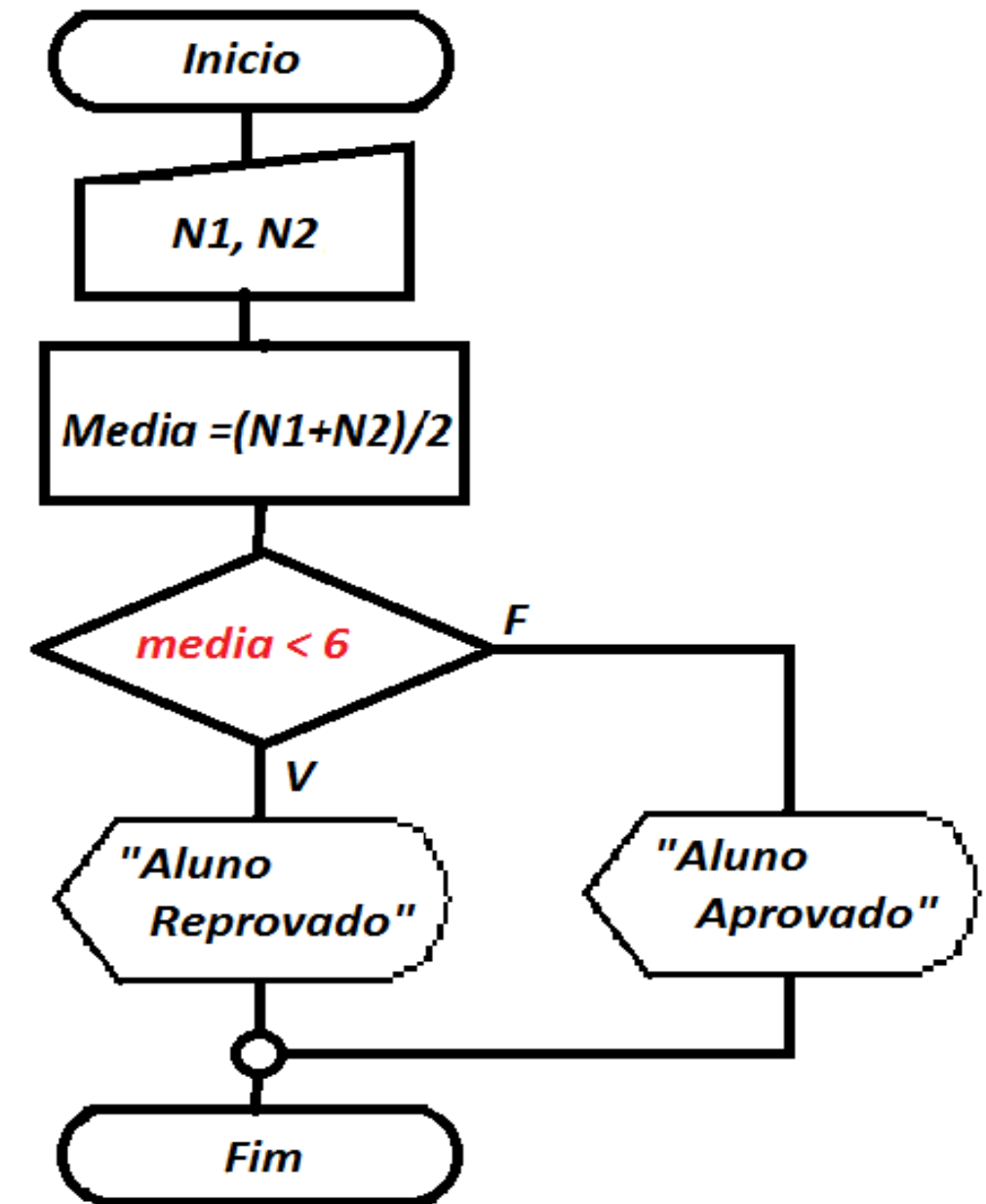
- Pode ficar extenso em algoritmos complexos
- Requer espaço para representação
- Dificulta modificações em algoritmos grandes

Exemplo de Fluxograma

Cálculo de Média de Duas Notas

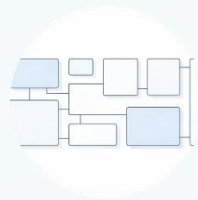
O fluxograma a seguir demonstra visualmente a lógica para calcular a média de duas notas e determinar se o aluno foi aprovado ou reprovado. Observe como os símbolos padronizados tornam o fluxo de execução claro e fácil de seguir.

Observe como o fluxograma permite visualizar rapidamente o caminho que o algoritmo seguirá dependendo do valor da média calculada. As setas indicam a direção do fluxo de execução, tornando a lógica condicional muito mais clara do que em texto puro.



Pseudocódigo

O pseudocódigo é uma forma de representação mais estruturada que se aproxima das linguagens de programação, mas mantém a legibilidade da linguagem natural. Ele utiliza palavras-chave convencionais como "início", "fim", "se", "então", "enquanto" para expressar a lógica do algoritmo. Em resumo é uma forma intermediária entre a linguagem natural e a linguagem de programação.



Estruturado e Claro

Possui uma estrutura bem definida com início, processamento e fim, facilitando a compreensão da lógica



Independente de Linguagem

Pode ser facilmente traduzido para qualquer linguagem de programação sem perder sua essência



Precisão e Detalhamento

Menos ambíguo que a narrativa, permitindo uma representação mais precisa da lógica algorítmica

Estrutura do Pseudocódigo

Elementos Fundamentais

Todo pseudocódigo possui elementos básicos que o estruturam e tornam a lógica clara e executável. Compreender esses elementos é essencial para criar algoritmos bem estruturados.

Exemplo de Estrutura

```
início  
  declaração de variáveis  
  entrada de dados  
  processamento  
  saída de dados  
fim
```

Palavras-Chave

início, fim, se, então, senão, enquanto, para, leia, escreva

Variáveis

Espaços de memória para armazenar dados durante a execução

Operadores

Símbolos para realizar operações matemáticas e lógicas (+, -, *, /, =, >, <)

Estruturas de Controle

Condicionais (se/então) e repetições (enquanto/para)

Vantagens e Limitações

Pseudocódigo

Vantagens ✓

- Mais organizado e estruturado
- Próximo da programação real
- Fácil de transformar em código
- Reduz ambiguidades
- Independente de linguagem específica

Limitações ⚠

- Exige conhecimento de estruturas algorítmicas
- Não é executável diretamente
- Pode variar em sintaxe entre diferentes autores

Exemplo Completo: Cálculo de Média

Pseudocódigo

```
var
    nota1, nota2, media: real

inicio

    escreva("Digite a primeira nota: ")
    leia(nota1)

    escreva("Digite a segunda nota: ")
    leia(nota2)

    media <- (nota1 + nota2) / 2

    se media >= 60 então
        escreval("Aprovado! Média: ", media)
    senão
        escreval("Reprovado! Média: ",
media)
    fimse

finalgoritmo
```

Análise do Algoritmo

Este algoritmo demonstra os principais elementos do pseudocódigo em ação:

- **Declaração de variáveis:** Define os espaços de memória necessários
- **Entrada de dados:** Solicita as duas notas ao usuário
- **Processamento:** Calcula a média aritmética
- **Estrutura condicional:** Verifica se o aluno foi aprovado
- **Saída:** Exibe o resultado final

A estrutura clara e lógica facilita a posterior conversão para qualquer linguagem de programação.

Linguagem de Programação

A linguagem de programação é a implementação real do algoritmo em uma sintaxe específica que pode ser executada pelo computador. É o estágio final do desenvolvimento, onde a lógica algorítmica se transforma em código executável.

Exemplo em PHP CLI

```
<?php
echo "Digite a primeira nota: ";
$nota1 = floatval(trim(fgets(STDIN)));
echo "Digite a segunda nota: ";
$nota2 = floatval(trim(fgets(STDIN)));
$media = ($nota1 + $nota2) / 2;
echo "\nNotas informadas:\n";
echo "Primeira nota: $nota1\n";
echo "Segunda nota: $nota2\n";
echo "Média calculada: " . number_format($media, 2, '.', '') . "\n";
if ($media >= 60) {
    echo "Resultado: Aprovado!\n";
} else {
    echo "Resultado: Reprovado!\n";
}
?>
```


Comparação e Aplicações

Descrição Narrativa (Linguagem Natural)

Melhor para: Comunicação inicial, brainstorming, explicação para não-programadores

Limitações: Ambiguidades, falta de precisão técnica

Pseudocódigo

Melhor para: Desenvolvimento de lógica, transição para código, documentação técnica

Limitações: Requer conhecimento de estruturas algorítmicas básicas

Fluxograma

Melhor para: Visualização do fluxo lógico, documentação visual, identificação de erros

Limitações: Pode ficar extenso, dificuldade de modificações em algoritmos grandes

Linguagem de Programação

Melhor para: Implementação executável, testes reais, produto final

Limitações: Dependente de linguagem específica, requer conhecimento técnico avançado

Recomendação Prática

Para um desenvolvimento eficaz, comece com a **descrição narrativa** para compreender o problema e comunicar a solução de forma geral. Em seguida, utilize o **fluxograma** para visualizar o fluxo lógico, refinando a solução com **pseudocódigo** para detalhar a lógica antes de finalmente **implementar em uma linguagem de programação**. Essa abordagem sequencial maximiza a clareza e a precisão no desenvolvimento de algoritmos.